



中山大學
SUN YAT-SEN UNIVERSITY

数据抽象和类

中山大学计算机学院



主讲人：郑贵锋

zhenggf@mail.sysu.edu.cn



中山大学MOOC课程组

C/C++程序内存布局与对象（变量）存放位置

正文

- 函数实现，库实现，字符串等资源。不可改

静态变量（对象）

- 编译阶段初始化数据
- 用零初始化

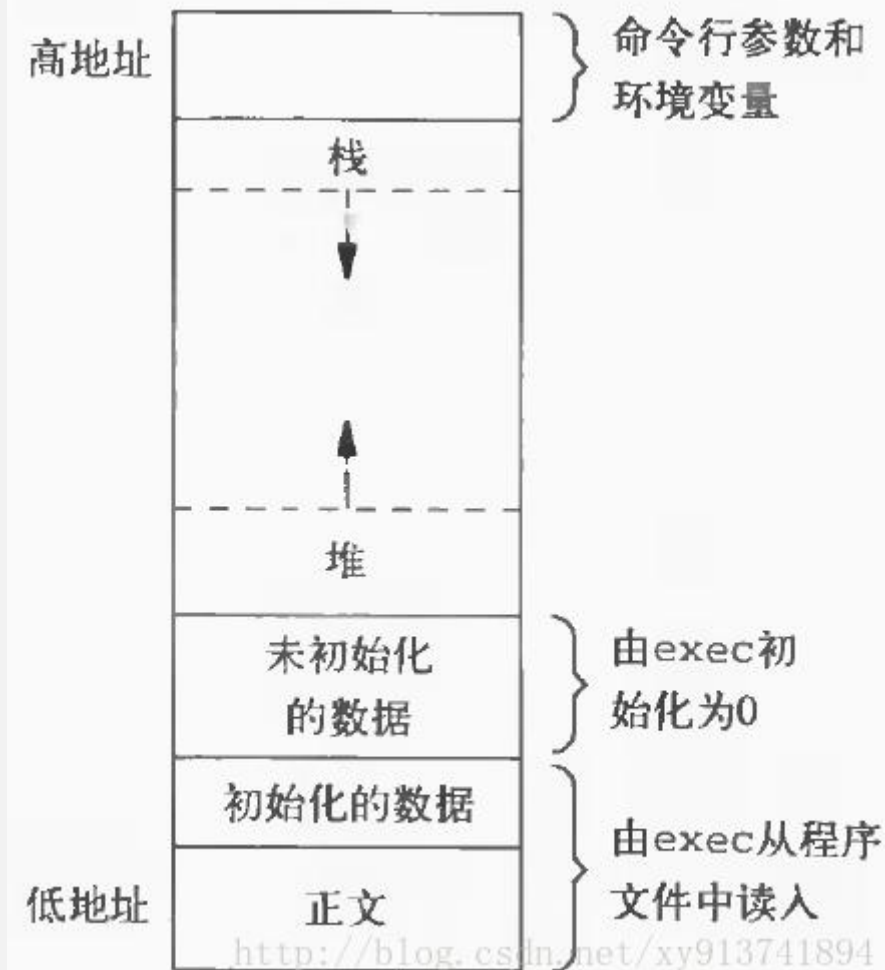
栈（Stack）

- 函数参数，自动变量（对象）

堆（Heap）

- 动态变量（对象），由 `stdlib.h` 管理

```
char *s1 = "Literal"; // 文字在代码区，仅分配了字符指针
// s1[0] = 'I'; // Segmentation fault
char s2[] = "Initial Literal"; // 分配数组空间
Point *po = &((Point){2,3}); // 分配结构空间且指针
```





C语言动态对象（变量）管理

堆（Heap）

- 共享的对象（变量）空间
- 由 `stdlib` 库管理

正确使用堆空间

- 必须 `#include <stdlib.h>`
- 申请空间, `void * malloc(size_t)`
- 释放空间, `void free(void *)`
- 申请的空间必须释放, 否则就是**内存泄漏**
- **Free 后再使用指针或释放**, 行为不可预测
- 如何申请、释放对象或一维数组?
- 如何申请、释放二维数组?

C++也是这样用的吗?

```
#include<stdio.h>
#include<stdlib.h>

typedef struct { int x; int y; } Point;

int main() {
    // 分配变量或一维可变数组
    Point *p1 = malloc(sizeof(Point) * 10);
    // 分配二维数组(n,m是常数)
    Point (*p2)[2][3] = malloc(sizeof(Point) * 6);
    // 分配数组的数组
    int n = 2, m = 3;
    Point **p3 = malloc(sizeof(Point *) * n);
    for (size_t i = 0; i < n; i++)
    {
        p3[i] = malloc(sizeof(Point) * m);
    }
    // do something
    free(p1);
    free(p2);
    // 必须先释放行数组
    for (size_t i = 0; i < n; i++)
    {
        free(p3[i]);
    }
    free(p3);
    return 0;
}
```



C++用 malloc 分配对象数组 (超纲)

程序要点

- `#include<cstdlib>`
- `if` 语句判断空间是否分配
- 初始化对象
 - `new(void *)` 构造函数
- 处理构造异常
- 调用对象析构
 - 对象.`~`析构函数
- 最后释放空间

```
#include <iostream>
#include <cstdlib>
#include <string>

int main() {
    // 为 4 个 string 的数组分配足够空间
    if(auto p = (std::string*)std::malloc(4 * sizeof(std::string))) {
        int i = 0;
        try {
            for(; i != 4; ++i) // 填充数组
                new(p + i) std::string(5, 'a' + i);

            for(int j = 0; j != 4; ++j) // 打印出来
                std::cout << "p[" << j << "] == " << p[j] << std::endl;
        }
        catch(...) {}

        for(; i != 0; --i) // 清理
            p[i - 1].~basic_string();

        std::free(p);
    }
}
```

C++新关键字：new 和 delete 运算符

新关键字优点：

- **new** 类型 初始化
 - 分配空间
 - 每个对象调用构造器
 - 有错误抛出异常
- **delete []p**
 - 为数组每个对象析构
 - 释放空间

Cppreference: new 表达式

```
/*new-delete-strings-demo*/
#include <iostream>
#include <string>

int main() {
    // 为 n 个 string 的数组分配空间
    int n = 4;
    std::string *p = new std::string[n] {
        std::string(5, 'a'),
        std::string(5, 'b'),
        std::string(5, 'c'),
        std::string(5, 'd')
    };

    for(int i=0; i<n; i++) {
        std::cout << p[i] << std::endl;
    }

    delete []p;
}
```

目录

CONTENTS

01

对象指针

02

=delete 和 =default

03

列表初始化

04

对象的内存布局

05

拷贝构造函数

06

传值和传引用

07

常引用\常方法\常量正确性



动态对象（变量）与对象指针

动态对象的使用：code1.cpp

- 申请
 - new 类型
 - 返回该类型的指针
 - 不是万能指针 void *
- 使用
 - *p 解析为对象
- 释放
 - delete 指针

```
#include<iostream>
using namespace std;

int main() {
    int *p;
    p = new int;

    cout << "Please enter an integer value: ";
    cin >> *p;
    cout << "The value you enter is: " << *p;

    delete p;
}
```

```
Please enter an integer value: 3
The value you enter is: 3
```




动态对象（变量）与对象指针

动态对象的使用：code2. cpp

- 申请数组
 - new 类型[] {初始化列表}
 - 和定义自动变量类似。没有初始化列表调用无参构造（默认构造一般是不确定值）；常数初始化列表，后面补零；长度小于初始化序列长度，抛出异常
 - 返回该类型的指针
- 使用
 - *p 直接看作数组使用
- 释放
 - delete []指针

```
#include<iostream>
using namespace std;

void inc(int *p, int Length) {
    for (int i = 0; i < Length; i++)
        p[i]++;
}

int main() {
    int *p;
    int Length, i;

    cout << "Enter the lenght you want: ";
    cin >> Length;
    p = new int[Length]{1,3,5};

    inc(p, Length);

    for (i = 0 ; i < Length; i++)
        cout << *(p+i) << " ";

    delete []p;
}
```




new运算符的使用

指针=**new** 类型名; //动态创建一个对象

指针=**new** 类型名 (初始化参数); //动态创建一个对象

指针=**new** 类型名[数组长度]; //用于动态分配数组

- 初始化参数及其括号为可选
- 类型可为基本类型，也可为类类型，若为类类型，则初始化参数相当于将实际参数传递给该类的构造函数
- **new**运算返回一个该类型指针（不是 **void***），指向分配到的内存空间
- 若内存分配失败，抛出异常，不是 **NULL**

对象指针访问成员 - 例子

```
class DATE
{
public:
    DATE( int initYear, int initMonth, int initDay ); //构造函数
    DATE(); //缺省构造函数
    void Set( int newMonth, int newDay, int newYear );
    int getMonth() const;
    int getDay() const;
    int getYear() const;
    void Print() const;
    void Increment();
    void Decrement();

private:
    int month;
    int day;
    int year;
};
```

//code3



例子

```
int main() {  
    DATE* date_ptr;  
    DATE  date(1999, 10, 1);  
  
    date_ptr = &date;  
  
    date.Print();  
    date_ptr->Print();  
}
```

取对象地址

October 1, 1999
October 1, 1999

对象date

date_ptr →

■ 对象取成员

-> 指针取成员

对象的动态创建和撤销

//请修改 code3 为 code4

```
DATE* date_ptr;  
date_ptr = new DATE(1976,  
12 ,20);
```

```
date_ptr -> Print();
```

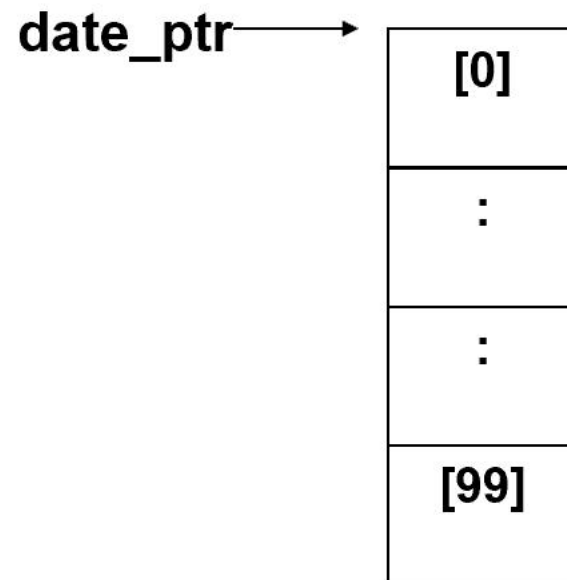
```
delete date_ptr;
```

//请修改 code3 为 code5

```
DATE* date_ptr;  
int k = 100;  
date_ptr = new DATE[k];
```

```
date_ptr -> Print();
```

```
delete [ ]date_ptr;  
//[ ]将令所有元素都调用各自的析构
```





delete操作符的使用

delete 变量名; //基本用法

delete []变量名; //用于释放数组

- 如果动态分配了一个数组，但是却用**delete p**的方式释放，没有用**[]**，则编译时没有问题，运行时也一般不会发生错误，但实际上会导致动态分配的数组没有被完全释放。
- **delete**释放的是指针所指对象占据的内存。
- 用**delete**释放空间后，指针的值仍是原来指向的地址，但指针已无效（重复释放将出错）。
- **delete**对象指针，会调用该对象的析构函数。

内存泄漏：常见情况

内存泄漏：指 **new** 的空间失去了指针或引用永远无法释放。导致资源耗尽！

指针赋值时可能发生：

```
int* p = new int(7); // 动态分配的 int 带值 7
p = nullptr; // 内存泄漏
```

指针离开作用域：

```
void f()
{
    int* p = new int(7);
} // 内存泄漏
```

异常导致程序或函数中止：

```
void f()
{
    int* p = new int(7);
    g(); // 可能抛出异常
    delete p; // 若无异常则 ok
} // 若 g() 抛出异常则内存泄漏
```

非法（野）指针使用错误

```
DATE* date_ptr;  
DATE* date_ptr2;  
date_ptr = new DATE(1976, 12 ,20);
```

```
date_ptr2 = date_ptr;
```

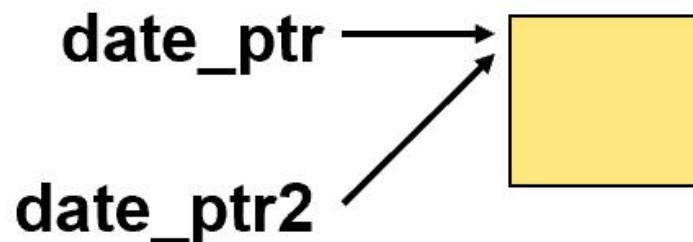
```
date_ptr -> Print();    //正确  
date_ptr2 -> Print();   //正确
```

```
delete date_ptr;        //将使date_ptr和date_ptr2指向失效的空间
```

```
date_ptr -> Print();    //错误，date_ptr已失效  
date_ptr2 -> Print();   //错误，date_ptr2也已失效
```

```
delete date_ptr2;       //错误，不能再次释放
```

//code6



非法（野）指针使用错误

```
//code 7  
//对于指向简单数据类型的指针，也有相同的问题。
```

```
int* p1;  
int* p2;
```

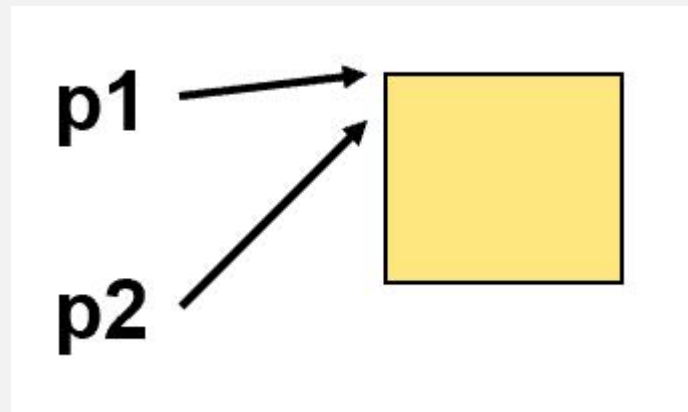
```
p1 = new int;  
p2 = p1;
```

```
*p1 = 10;  
cout << *p2 << endl;
```

```
delete p1;          //将使p1r和p2同时失效
```

```
cout << *p1;        //错误，p1已失效  
cout << *p2;        //错误，p2也已失效
```

```
delete p2;          //错误，不能再次释放
```





显式默认化的函数定义：=default

```
#include <iostream>
using namespace std;

class A
{
public:
    A(int x) {
        cout << "This is a parameterized constructor";
    }
    A() = default;
};

int main(){
    A a;           //call A()
    A x(1);        //call A(int x)
    cout<<endl;
    return 0;
}
```

令编译器为某个类生成特殊成员函数或比较运算符的显式指令。

(C++11)

- 当我们声明有参构造函数时，编译器就不会创建默认构造函数。
- =default 必须在函数申明后，让编译创建该构造函数
- 特殊成员函数包括：
 - 默认构造函数
 - 析构造函数
 - 复制构造函数
 - 等
- 函数不能写实现

//code8



=default

默认函数需要用于特殊的成员函数

```
class B
{
public:
    // Error, func is not a special member function.
    int func() = default;

    // Error, constructor B(int, int) is not a special member function.
    B(int, int) = default;

    // Error, constructor B(int=0) has a default argument.
    B(int = 0) = default;
};
```



弃置函数: =delete

```
class A {  
public:  
    A(int x): m(x) { }  
  
    // Delete the copy constructor  
    A(const A&) = delete;  
    // Delete the copy assignment operator  
    A& operator=(const A&) = delete;  
    int m;  
};
```

```
int main() {  
    A a1(1), a2(2), a3(3);  
    // Error, the usage of the copy assignment operator is disabled  
    a1 = a2;  
    // Error, the usage of the copy constructor is disabled  
    a3 = A(a2);  
    return 0;  
}
```

如果取代函数体而使用特殊语法 =

 ;, 则该函数被定义为弃置的
()。

任何弃置函数的使用都是非良构的（程序无法编译）。这包含调用，包括显式（以函数调用运算符）及隐式（对弃置的重载运算符、特殊成员函数、分配函数等的调用），构成指向弃置函数的指针或成员指针，甚至是在不求值表达式中使用弃置函数。

//code10



=delete

请注意，删除的函数是隐式内联的，这一点非常重要。删除的函数定义必须是函数的**首次声明**。

以下方法是将函数声明为已删除的正确方法：

```
class C {  
public:  
    C(C& a) = delete;  
};
```

但是以下尝试声明删除函数的方法会产生错误：

```
// incorrect syntax of declaring a member function as deleted  
class C {  
public:  
    C();  
};
```

```
// Error, the deleted definition of function C must be the first declaration of the function.  
C::C() = delete;
```

案例研究：链表 (Linked Table)

- 链表是一种数据结构用于存储序列数据
- 链表的特征
 - 有一个指针指向 LinkNode 类型对象
 - LinkNode对象包含指向下一个的指针
 - 直到 NULL
- LinkNode 类型
 - 包含数据成员保存用户数据
 - LinkNode 类型指针
- 数组不好？LinkNode * 多浪费空间？

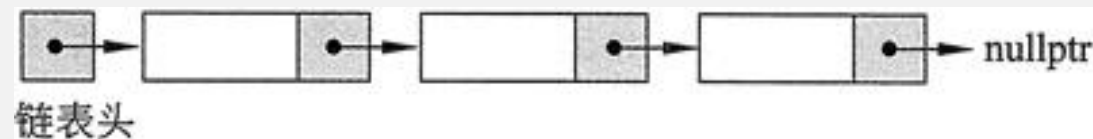


图 1 链表结构图解

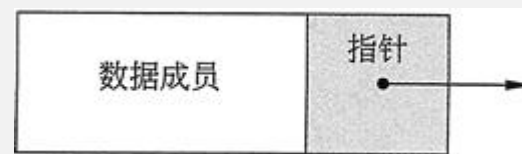


图 2 链表结点的结构



案例研究：链表 – 类声明

link-node.hpp

- 请**正确书写头文件**，避免循环嵌套
- 使用 typedef 使得程序更通用
- = default
- 注意 const
- static, 为什么?

```
#ifndef LINK_NODE_H
#define LINK_NODE_H
//你的代码写在这里
typedef int UserData;

class LinkNode {
public:
    LinkNode() = default;
    LinkNode(UserData item): item(item) {};
    UserData getData() const;
    LinkNode* getNext() const;
    void setNext(LinkNode *next);

    static LinkNode* Push(LinkNode *head, UserData item);
private:
    UserData item;
    LinkNode *next;
};

#endif
```



案例研究：链表 – 类实现

link-node.cpp

- Get/Set 成员很简单
- 注意 this
- 注意 static 成员函数不能访问类的普通成员数据和函数
- auto 的使用
- new 关键字
- head 是指针, 传引用
- 函数可以不用返回 head。对吗?

```
#include "linknode.hpp"

UserData LinkNode::getData() const {
    return item;
}

LinkNode* LinkNode::getNext() const {
    return next;
}

void LinkNode::setNext(LinkNode *next) {
    this->next = next;
}

LinkNode* LinkNode::Push(LinkNode *head, UserData item) {
    auto temp = head;
    head = new LinkNode(item);
    head->setNext(temp);
    return head;
}
```




案例研究：链表 – 应用

link-example.cpp

- Head 必须负初值为 NULL
- 静态成员作为对象成员函数访问
- 作为类域下的静态成员
- 生成的链表是什么样子？
 - head 指向的节点用户数据？
 - 最后一个节点用户数据？
- 为什么不用 head 直接控制循环
- 这个循环功能是什么？
- 没有 “temp = temp->getNext();” 会如何？

```
#include<iostream>
#include"linknode.hpp"

int main() {
    LinkNode *head = NULL;
    head = head->Push(head,3);
    head = LinkNode::Push(head,7);
    head = LinkNode::Push(head,25);
    head = LinkNode::Push(head,4);

    auto temp = head;
    while (temp != NULL) {
        std::cout << temp->getData() << std::endl;
        temp = temp->getNext();
    }
    return 0;
}
```



课后练习：链表 – 扩展功能

请修改程序，添加一个 `pop(head)` 成员函数返回 `LinkNode *`。

该函数是删除`head`指向的节点，并使`head`指向下一个节点。

再 `main` 函数中 `push`, `pop` 若干次，观察输出结果



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组